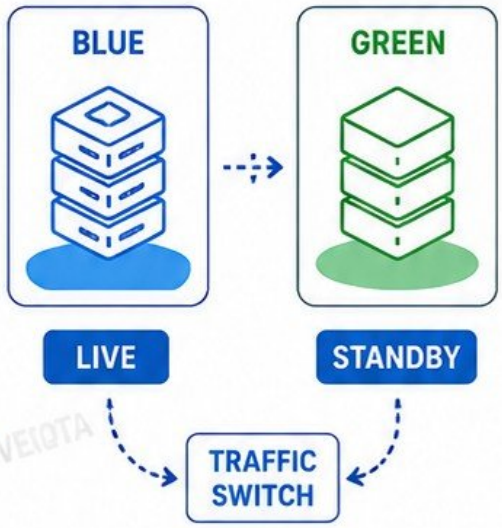
VERIQTA

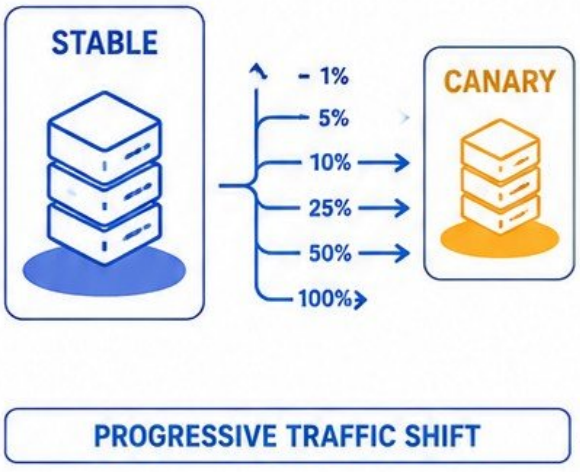
BLUE-GREEN & CANARY DEPLOYMENT HANDBOOK

STRATEGIES | ARCHITECTURES | BEST PRACTICES | PRODUCTION READY

BLUE-GREEN DEPLOYMENT



CANARY DEPLOYMENT



REDUCE RISK | IMPROVE RELIABILITY | INCREASE OBSERVABILITY | AUTOMATE SAFELY | ROLLBACK INSTANTLY

ZERO DOWNTIME. MAXIMUM CONFIDENCE.
DELIVER BETTER. RELEASE SAFER.

/VERIQTA /VERIQTA /VERIQTA /VERIQTA /VERIQTA /VERIQTA

PAGE 1

VERIQTA

DEPLOYMENT RISK IN PRODUCTION

UNDERSTAND THE RISKS. DEPLOY WITH CONFIDENCE.

1 WHY DEPLOYMENTS FAIL AFTER SUCCESSFUL CI/CD

- Tests don't cover real-world scenarios
- Configuration differences in production
- Database schema or data issues
- External service dependencies fail
- Insufficient observability and alerts
- Traffic patterns differ from staging
- Hidden performance bottlenecks
- Human error during deployment
- Incomplete rollback strategy
- Unmanaged infrastructure drift

2 RELEASE RISK VS INFRASTRUCTURE RISK

RELEASE RISK (APPLICATION)

- New code defects
- Logic errors
- Performance regressions
- Integration failures
- Feature misbehavior

INFRASTRUCTURE RISK (PLATFORM)

- Resource exhaustion
- Network or DNS issues
- IAM / permission misconfigurations
- Infrastructure changes
- Third-party service outages

3 BLAST RADIUS

The potential impact area of a failed deployment.
Smaller blast radius = lower risk.

How to reduce:

- Region isolation
- AZ isolation
- Service isolation
- Canary or blue-green strategies
- Feature flags and gradual rollout

4 TRAFFIC EXPOSURE

How much real user traffic is exposed to the new deployment.

Best practices:

- Start small (1% or less)
- Increase gradually
- Monitor key metrics
- Automate promotion
- Stop on anomaly

5 DEPLOYMENT VS RELEASE

DEPLOYMENT: Code is delivered to production environment.

RELEASE: Code is made available to users.

Every deployment is **NOT** a release.
Control exposure to reduce risk.

6 ROLLBACK VS ROLL-FORWARD

ROLLBACK

- Revert to previous stable version
- Fast but not always safe

ROLL-FORWARD

- Deploy a new version that fixes the issue
- Preferred for complex changes

7 PRODUCTION DEPLOYMENT DECISION FLOW

1. **DEFINE OBJECTIVE** — What are we trying to achieve?
2. **ASSESS RISK** — What can go wrong? What is the blast radius?
3. **CHOOSE STRATEGY** — Blue-Green, Canary, or Rolling?
4. **PRE-DEPLOY VALIDATION** — Test, verify, and confirm readiness
5. **DEPLOY & MONITOR** — Deploy, monitor metrics, logs, and alerts
6. **DECIDE & ACT** — Promote, Rollback, or Roll-Forward based on data

DEPLOY SMART. MONITOR CONTINUOUSLY. REDUCE RISK. DELIVER VALUE.

PAGE 2

VERIQTA

BLUE-GREEN DEPLOYMENT ARCHITECTURE

ZERO DOWNTIME. INSTANT ROLLBACK. MAXIMUM SAFETY.

1 BLUE AND GREEN ENVIRONMENTS

- Two identical production environments.
- BLUE = current live environment.
- GREEN = new version ready for production.
- Only one environment serves live traffic at a time.

2 PRODUCTION TRAFFIC ROUTING

- All user traffic goes through a single entry point.
- Traffic is routed to either BLUE or GREEN.
- No direct access to environments.

3 LOAD BALANCER AND DNS SWITCHING

- DNS or Load Balancer points to the active environment.
- Switch is instant and reversible.
- Health checks ensure only healthy environment receives traffic.

4 ENVIRONMENT SYNCHRONIZATION

- Infrastructure, config, and secrets kept in sync.
- Database schema compatible across both.
- Shared services and integrations identical.
- Use IaC to ensure consistency.

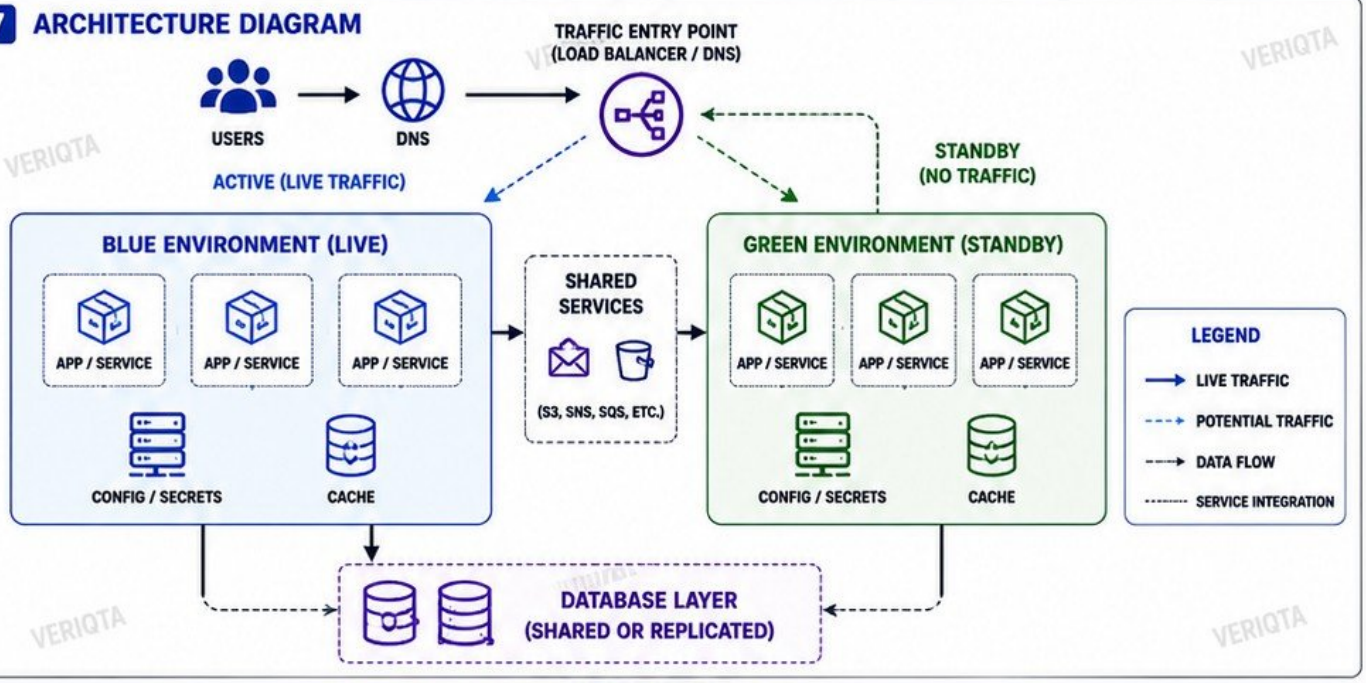
5 DATABASE DEPENDENCIES

- Use a shared database or a replicated setup.
- Schema changes must be backward compatible.
- Avoid destructive changes until after cutover.
- Consider read replicas for scaling and safety.

6 CUTOVER SEQUENCE

1. Deploy and test GREEN environment.
2. Validate functionality and performance.
3. Warm up GREEN (cache, connections, jobs).
4. Switch traffic from BLUE to GREEN.
5. Monitor GREEN closely.
6. Retire BLUE after stability confirmation.

7 ARCHITECTURE DIAGRAM



KEY BENEFITS

ZERO DOWNTIME | INSTANT ROLLBACK | REDUCED RISK | HIGH RELIABILITY | EASY AUTOMATION | PRODUCTION SAFETY

PAGE 3

VERIQTA

BLUE-GREEN DEPLOYMENT LIFECYCLE

SAFE RELEASES. INSTANT ROLLBACK. ZERO DOWNTIME.

		STATE	KEY ACTIONS & CHECKS
1	**PROVISION GREEN ENVIRONMENT** Create an identical environment (infrastructure, config, services, and dependencies) for the new version.	LIVE TRAFFIC → BLUE (LIVE) ⇢ GREEN (PROVISIONING)	• Clone infrastructure and config • Sync secrets and integrations • Verify networking and access • Infrastructure health checks
2	**DEPLOY NEW RELEASE** Deploy the new application version to the GREEN environment.	LIVE TRAFFIC → BLUE (LIVE) ⇢ GREEN (NEW RELEASE)	• Build and deploy new version • Run database migrations (safe) • Warm up caches and connections • Verify deployment success
3	**VALIDATE BEFORE EXPOSURE** Ensure the GREEN environment is healthy and ready before receiving any production traffic.	LIVE TRAFFIC → BLUE (LIVE) ⇢ GREEN (READY)	• Health checks • Dependency checks • Config and feature flags • Resource and capacity checks
4	**RUN SMOKE AND INTEGRATION TESTS** Run automated smoke tests and key integration tests against GREEN.	LIVE TRAFFIC → BLUE (LIVE) ⇢ GREEN (TESTING)	• Smoke tests (critical paths) • Integration tests • API and contract tests • Performance baseline checks
5	**SWITCH PRODUCTION TRAFFIC** Switch traffic from BLUE to GREEN using the load balancer or DNS.	BLUE (STANDBY) → GREEN (LIVE) ← LIVE TRAFFIC	• Update load balancer target group • Or update DNS / routing • Verify traffic switch • Monitor error rate and latency
6	**OBSERVE THE NEW ENVIRONMENT** Closely monitor GREEN after the cutover to ensure stability and performance.	BLUE (STANDBY) → GREEN (LIVE) ← LIVE TRAFFIC	• Monitor metrics, logs, and traces • Check KPIs and business metrics • Watch alarms and dashboards • Confirm no regression
7	**RETAIN OR TERMINATE BLUE** Keep BLUE as a fallback for a defined period or terminate it to save cost.	BLUE (RETIRING) → GREEN (LIVE) ← LIVE TRAFFIC	• Decide retention window • Take final backup if needed • Decommission or scale down • Clean up resources
8	**ROLLBACK DECISION POINTS** At any point, if issues are detected, rollback instantly to BLUE.	BLUE (STANDBY) ⇠ GREEN (LIVE) ← LIVE TRAFFIC	• Error rate above threshold • Performance degradation • Failed health checks • Business KPI impact

ROLLBACK FLOW (ANY TIME BEFORE OR AFTER TRAFFIC SWITCH)

ISSUE DETECTED → REVERT ROUTING TO BLUE → BLUE (LIVE) → VERIFY STABILITY → INVESTIGATE ROOT CAUSE → FIX & REDEPLOY TO GREEN

BEST PRACTICES

AUTOMATE EVERY STEP | VALIDATE BEFORE EXPOSURE | MONITOR CONTINUOUSLY | KEEP ROLLBACK SIMPLE | OPTIMIZE COST AFTER RELEASE | DOCUMENT AND IMPROVE

PAGE 4

VERIQTA

CANARY DEPLOYMENT ARCHITECTURE

SMALL EXPOSURE. CONTINUOUS VALIDATION. SAFE PROGRESSION.

1 STABLE VS CANARY VERSIONS

STABLE

- Current proven version
- Handles majority of traffic

CANARY

- New version
- Handles small percentage of traffic

2 PROGRESSIVE TRAFFIC EXPOSURE

- Gradually increase traffic to the canary.
- Continuously validate metrics at each step.

3 TRAFFIC SHIFT STEPS

1% → 5% → 10% → 25% → 50% → 100%

4 WEIGHTED ROUTING

- Traffic is split based on defined weights.
- Easy to adjust and revert.
- Supported by ALB, Ingress, Service Mesh, API Gateway, DNS, and more.

5 CANARY COHORTS

- Send canary traffic to specific user groups.
- By user ID, geography, device, or behavior.
- Reduce risk and gain better feedback.

6 AUTOMATED PROMOTION

- Automatically promote when metrics are healthy.
- Remove manual delays.
- Save time and reduce human error.

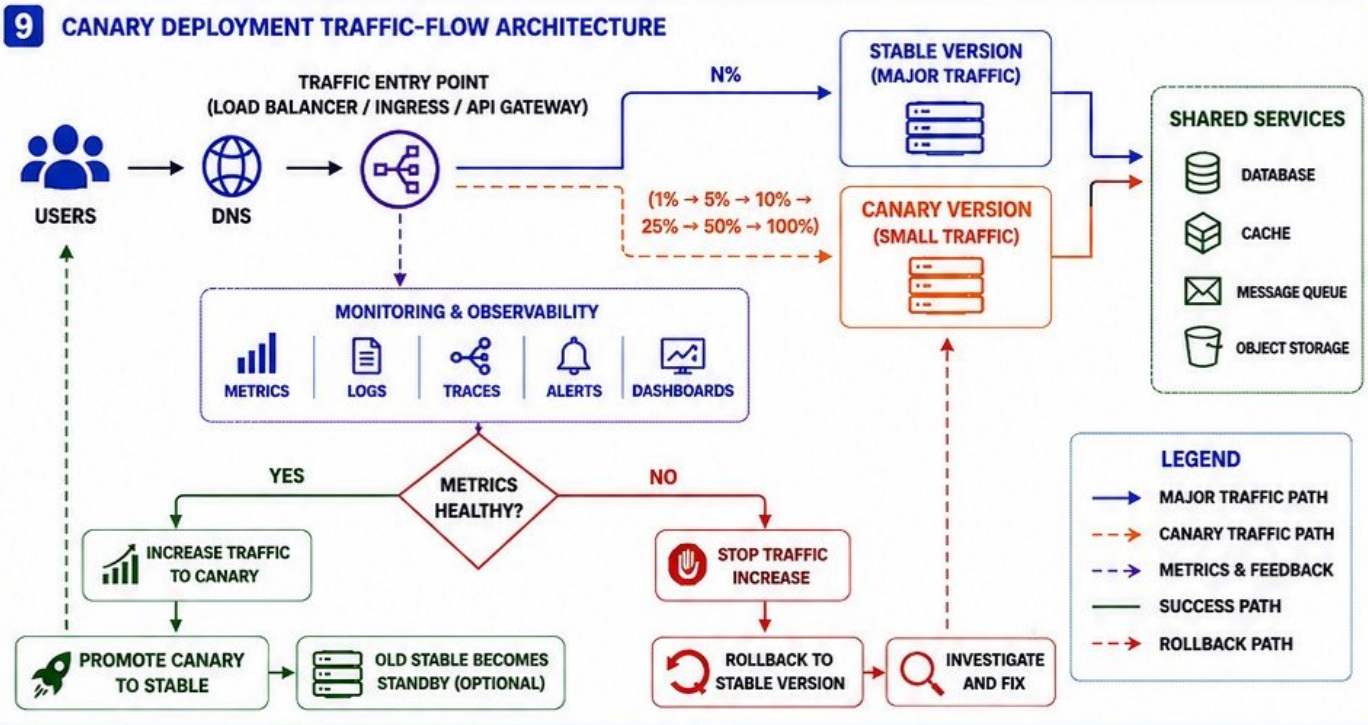
7 FAILURE THRESHOLDS

- Define thresholds for errors, latency, and other metrics.
- If breached, stop progression.
- Automatically rollback to stable version.

8 TRAFFIC-FLOW DIAGRAM

- See full architecture and traffic flow below.

9 CANARY DEPLOYMENT TRAFFIC-FLOW ARCHITECTURE



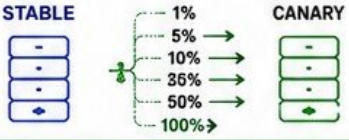
KEY BENEFITS

REDUCED RISK | EARLY ISSUE DETECTION | HIGHER RELIABILITY | AUTOMATED SAFETY | BETTER USER EXPERIENCE | FAST ROLLBACK

VERIQTA

BLUE-GREEN VS CANARY

UNDERSTAND THE DIFFERENCES. CHOOSE THE RIGHT STRATEGY.

CRITERIA	BLUE-GREEN DEPLOYMENT 	CANARY DEPLOYMENT 
RELEASE SPEED	FAST CUTOVER AFTER VALIDATION	SLOWER (PROGRESSIVE TRAFFIC SHIFT)
BLAST RADIUS	HIGH (100% TRAFFIC SWITCH)	LOW (LIMITED % OF TRAFFIC)
INFRASTRUCTURE COST	HIGH (TWO FULL ENVIRONMENTS)	LOW TO MEDIUM (EXTRA CAPACITY ONLY)
ROLLBACK COMPLEXITY	LOW (INSTANT TRAFFIC SWITCH BACK)	MEDIUM (TRAFFIC REDUCTION + ROLLBACK)
OBSERVABILITY REQUIREMENTS	MEDIUM (VALIDATE BEFORE CUTOVER)	HIGH (CONTINUOUS METRICS ANALYSIS)
STATEFUL WORKLOAD CONSIDERATIONS	EASIER (ONE VERSION LIVE AT A TIME)	HARDER (MIXED VERSIONS RECEIVE TRAFFIC)
WHEN EACH STRATEGY WINS	• NEED FAST, FULL RELEASES • EASY TO VALIDATE BEFORE CUTOVER • STATEFUL OR SESSION-SENSITIVE APPS • SMALL TEAMS, SIMPLE WORKFLOWS	• NEED SAFEST, RISK-MITIGATED RELEASES • COMPLEX SYSTEMS WITH HIGH RISK • LARGE USER BASE WITH VARIED BEHAVIOR • SLO AND ERROR-BUDGET DRIVEN RELEASES

DEPLOYMENT DECISION MATRIX

QUESTION	LEAN BLUE-GREEN 	LEAN CANARY 
DO YOU NEED FAST, ALL-AT-ONCE RELEASES?	YES	NO
CAN YOU FULLY VALIDATE BEFORE EXPOSING TRAFFIC?	YES	NO
IS THE APPLICATION HIGH RISK OR CUSTOMER-FACING?	NO	YES
DO YOU HAVE STRONG OBSERVABILITY AND AUTOMATION?	BASIC IS ENOUGH	ADVANCED REQUIRED
IS THE WORKLOAD STATEFUL OR SESSION-SENSITIVE?	YES	NO
IS INFRASTRUCTURE COST A MAJOR CONCERN?	NO	YES

KEY TAKEAWAY

THERE IS NO ONE-SIZE-FITS-ALL.
CHOOSE THE STRATEGY THAT MATCHES YOUR RISK,
SYSTEM COMPLEXITY, TEAM MATURITY, AND BUSINESS GOALS.

PAGE 6

VERIQTA

TRAFFIC MANAGEMENT

ROUTE THE RIGHT TRAFFIC. DELIVER THE RIGHT EXPERIENCE.

SMART ROUTING | HIGH AVAILABILITY | SAFE RELEASES | BETTER USER EXPERIENCE | OPTIMIZED PERFORMANCE

1 LAYER 4 VS LAYER 7 ROUTING

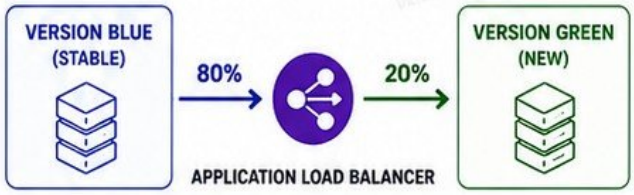
LAYER 4 (TRANSPORT)

- Routes based on IP address and port
- Fast and efficient
- No inspection of content
- Used for TCP/UDP traffic

LAYER 7 (APPLICATION)

- Routes based on content (HTTP/HTTPS)
- Can inspect headers, path, cookies, queries
- More flexible
- Used for advanced routing

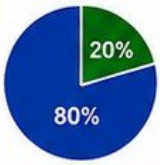
2 LOAD BALANCER TRAFFIC SHIFTING



TRAFFIC SHIFT MODES

- ALL AT ONCE (BLUE → GREEN)
- WEIGHTED (e.g., 90/10, 50/50)
- GRADUAL (progressive increase)
- CONDITIONAL (based on metrics)

3 WEIGHTED ROUTING



- 80% → BLUE (STABLE)
- 20% → GREEN (CANARY)

USE CASES

- Canary deployments
- A/B testing
- Gradual rollouts

4 HEADER-BASED ROUTING

HTTP REQUEST
Host: app.example.com
X-Env: canary
User-Agent: Chrome

→ IF X-Env = canary → SEND TO GREEN

→ IF X-Env != canary → SEND TO BLUE

COMMON USE CASES

- Internal users to new version
- QA or tester traffic
- Feature-based routing

5 COOKIE-BASED ROUTING

HTTP REQUEST
Cookie: version=green
User: 12345

→ IF Cookie = green → SEND TO GREEN

→ IF Cookie != green → SEND TO BLUE

BENEFITS

- Consistent experience per user
- Maintains session affinity
- Useful for canary testing

6 USER SEGMENTATION

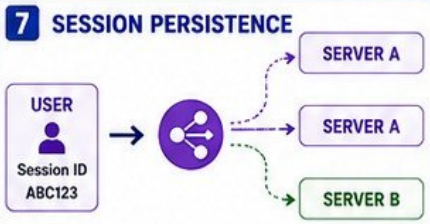
- NEW USERS
- RETURNING USERS
- GEOGRAPHY
- DEVICE TYPE
- PREMIUM USERS

→ ROUTE DIFFERENT SEGMENTS TO DIFFERENT VERSIONS OR EXPERIENCES

WHY IT MATTERS

- Target the right users
- Reduce risk
- Improve adoption

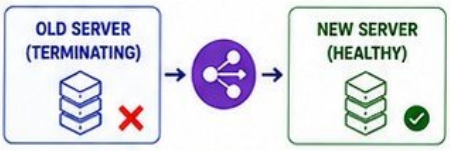
7 SESSION PERSISTENCE



IMPORTANT

- Keeps a user on the same server
- Prevents session loss
- Use cookies or source IP

8 CONNECTION DRAINING



HOW IT WORKS

- Stop sending new requests to old server
- Allow existing connections to complete
- Then terminate old server

9 AVOIDING UNEVEN TRAFFIC DISTRIBUTION

USE HEALTH CHECKS | PROPER WEIGHT CONFIGURATION | AVOID TOO MANY RULES | MONITOR TRAFFIC DISTRIBUTION | DETECT SKEW EARLY | AUTOMATE ADJUSTMENTS

BEST PRACTICES

- ✓ Continuously monitor metrics
- ✓ Validate routing rules
- ✓ Use consistent hashing when needed
- ✓ Test routing before full rollout
- ✓ Document and review rules

PAGE 7

VERIQTA

KUBERNETES BLUE-GREEN DEPLOYMENTS

TWO ENVIRONMENTS. INSTANT TRAFFIC SWITCH. ZERO DOWNTIME.

ZERO DOWNTIME | SAFE RELEASES | INSTANT ROLLBACK | FULL OBSERVABILITY | PRODUCTION READY

1 SEPARATE DEPLOYMENTS

- Create two Deployments: blue (stable) and green (new)
- Both run identical applications with different versions

Deployment/blue | Deployment/green

2 BLUE AND GREEN REPLICASETS

ReplicaSet (blue) v1.0.0 | ReplicaSet (green) v2.0.0

- Each Deployment creates its own ReplicaSet
- Kubernetes manages pods independently

3 READINESS PROBES

- Use Readiness probes to ensure pods are healthy
- Traffic is only sent to ready pods
- Prevents bad versions from receiving traffic

4 PRE-CUTOVER VALIDATION

- Run smoke tests and health checks against green
- Validate functionality, performance, and dependencies
- Ensure green is 100% ready

5 SERVICE SELECTOR SWITCHING

Service selector: app: blue → Service selector: app: green

- Service selector determines which pods receive traffic
- Switch selector from blue to green to cutover traffic

6 SERVICE CUTOVER

Users → Service (app) selector: app: green → ReplicaSet (green)

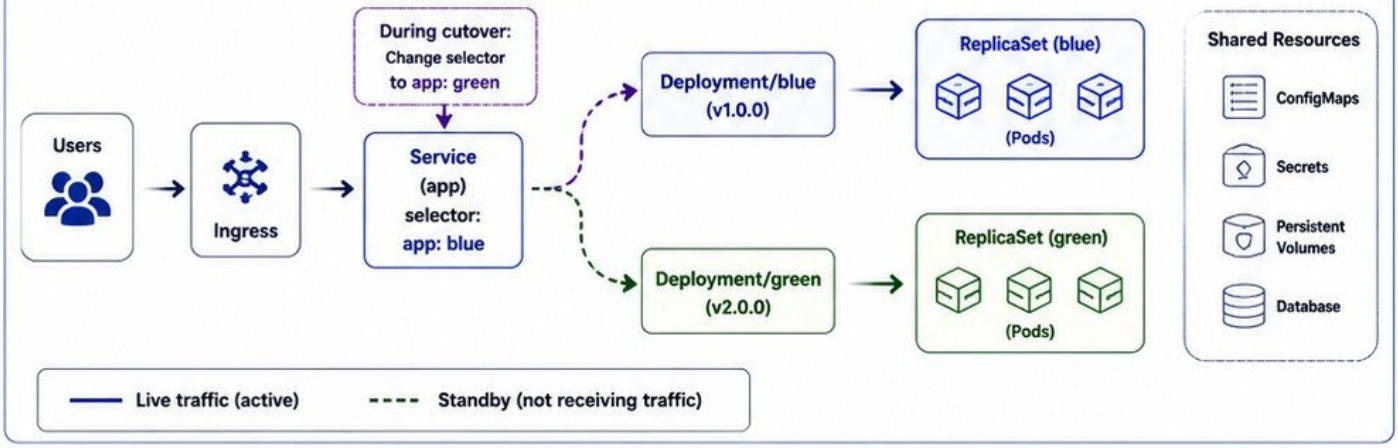
- All traffic now goes to green pods
- Blue remains running for quick rollback
- Monitor metrics, logs, and alerts

7 ROLLBACK BY SELECTOR REVERSAL

Service selector: app: green → Service selector: app: blue

- If issues occur, switch selector back to blue
- Instant rollback with zero downtime
- No redeploy required

KUBERNETES BLUE-GREEN ARCHITECTURE DIAGRAM



KEY TAKEAWAYS

- Run two environments in parallel: blue (stable) and green (new)
- Validate green thoroughly before sending any traffic
- Switch Service selector to cutover instantly
- Rollback instantly by switching selector back
- Monitor continuously after cutover

PAGE 8

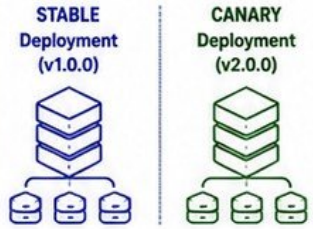
VERIQTA

KUBERNETES CANARY DEPLOYMENTS

PROGRESSIVE TRAFFIC. VALIDATE CONTINUOUSLY. REDUCE RISK.

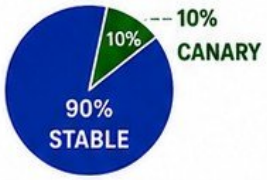
REDUCE RISK | VALIDATE REAL TRAFFIC | AUTOMATE PROGRESSION | INSTANT ROLLBACK | BETTER OBSERVABILITY

1 STABLE AND CANARY DEPLOYMENTS



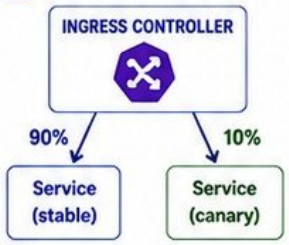
- Stable serves most traffic
- Canary serves a small portion
- Both run in parallel

2 REPLICA-BASED TRAFFIC APPROXIMATION



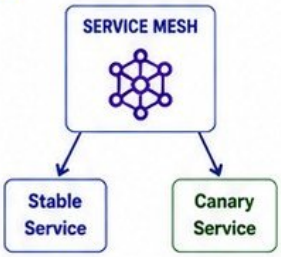
- Adjust canary replica count
- More replicas = more traffic
- Simple but not precise
- Good for internal traffic

3 INGRESS TRAFFIC SPLITTING



- Split traffic at Layer 7 (HTTP/HTTPS)
- Precise control with weights
- Path, host, header or cookie rules
- Works with NGINX, Traefik, Istio, etc.

4 SERVICE MESH ROUTING



- Advanced routing with Istio/Linkerd
- Fine-grained traffic control
- Retries, timeouts, fault injection
- Rich metrics per version

5 PROGRESSIVE DELIVERY

- Start with small traffic (e.g., 1%)
- Analyze metrics at each step
- Increase traffic gradually
- Stop or rollback on issues
- Reach 100% after successful validation

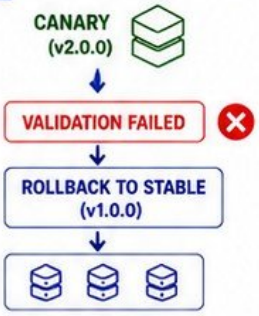
6 READINESS AND HEALTH GATES

- Readiness probes ensure pods are ready before receiving traffic
- Liveness probes detect unhealthy pods
- Health checks and SLO/metric gates control progression
- Prevent bad versions from impacting users

7 CANARY PROMOTION

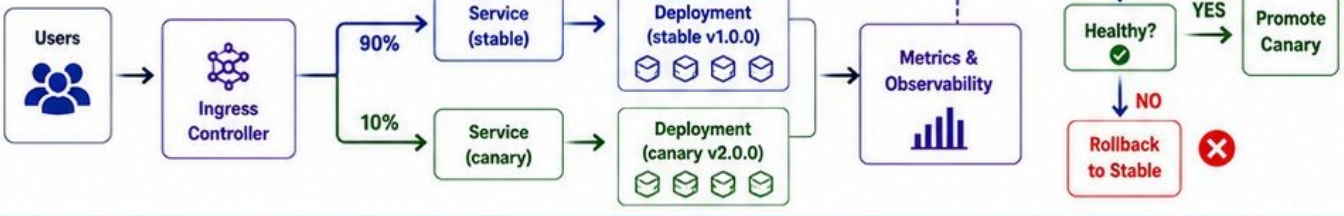
- Promote canary to stable
- Scale down old version
- Clean up old resources

8 FAILED-CANARY ROLLBACK



- Stop traffic to canary
- Keep stable serving all traffic
- Investigate and fix issues

KUBERNETES CANARY DEPLOYMENT FLOW



KEY TAKEAWAYS

- Run stable and canary in parallel
- Shift traffic progressively
- Validate with metrics and health gates
- Promote automatically when healthy
- Rollback instantly if issues occur
- Deliver new features safely

VERIQTA

AWS BLUE-GREEN DEPLOYMENTS

ZERO DOWNTIME. INSTANT ROLLBACK. PRODUCTION SAFE.

ZERO DOWNTIME | SAFE RELEASES | FAST ROLLBACK | AUTOMATED | PRODUCTION READY

1 CODEDEPLOY

AWS CodeDeploy

- Orchestrates blue-green deployments
- Validates green environment
- Manages traffic shifting and rollback
- Integrates with ALB, ECS, Lambda, Auto Scaling

2 APPLICATION LOAD BALANCER

- Routes traffic to target groups
- Supports weighted target group routing
- Health checks ensure only healthy targets receive traffic
- Enables instant traffic switch

3 TARGET GROUPS

- Blue target group (current)
- Green target group (new)
- ALB forwards traffic based on weights
- Health checks at target group level

4 AUTO SCALING GROUPS

- Separate ASGs for blue and green
- Ensures desired capacity in each environment
- Supports automatic scaling policies
- Replaced or retained after deployment

5 ECS BLUE-GREEN DEPLOYMENTS

- CodeDeploy + ECS service with task sets
- Blue task set (current)
- Green task set (new)
- Traffic shifted via ALB
- Zero downtime deployments

6 LAMBDA ALIASES

- Publish new version
- Create alias (green)
- Shift traffic using alias weights
- Instant rollback by shifting alias back

7 TRAFFIC SHIFTING

BLUE (TG) → GREEN (TG)

- All-at-once or linear/Canary shift
- Example: 10% → 25% → 50% → 100%
- Monitor metrics at every step
- Complete shift when healthy

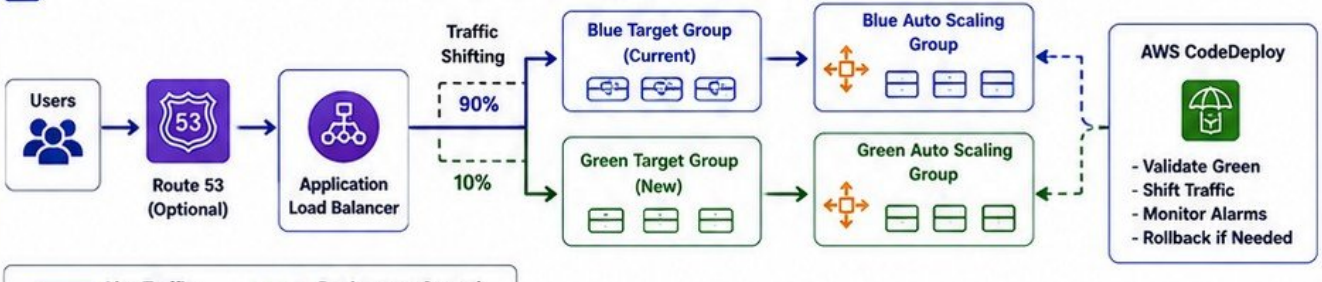
8 DEPLOYMENT GROUPS

- Logical group of targets (instances, ECS tasks, Lambda functions)
- Define blue & green target groups
- Configure traffic shifting strategy
- Set alarms and rollback triggers
- Control deployment lifecycle

9 AUTOMATED ROLLBACK

- CloudWatch alarms detect issues
- CodeDeploy automatically rolls back traffic
- Traffic returns to blue environment
- No manual intervention required

10 AWS BLUE-GREEN DEPLOYMENT ARCHITECTURE (ECS EXAMPLE)



BLUE-GREEN FLOW

1. Deploy new version to Green environment
2. Validate Green (tests, health checks)
3. Shift traffic from Blue to Green
4. Monitor metrics and alarms
5. Complete shift when stable
6. Retain or terminate Blue environment
7. Instant rollback by shifting traffic back to Blue

KEY BENEFITS

- Zero downtime deployments
- Instant rollback
- Reduced risk
- Consistent user experience
- Automated and repeatable

USE CASES

- Web applications
- APIs and microservices
- ECS services
- Lambda functions
- Mission-critical workloads

VERIQTA

AWS CANARY DEPLOYMENTS

SMALL TRAFFIC. CONTINUOUS VALIDATION. SAFE RELEASES.

REDUCE RISK | VALIDATE REAL TRAFFIC | PROGRESSIVE DELIVERY | AUTOMATED ROLLBACK | BETTER OBSERVABILITY

1 LAMBDA VERSIONS

- Each deployment creates a new version
- Immutable and unique
- Versions cannot be changed
- Enables safe traffic shifting
- Example: v1, v2, v3

v1 --> v2 --> v3

NEW VERSIONS OVER TIME

2 LAMBDA ALIASES

- Alias points to a version
- Route traffic using weights
- Update alias to shift traffic
- Enables instant rollback
- Example: alias "live"

ALIAS: live
WEIGHTS
v1 (90%)
v2 (10%)

→ VERSION v1 (STABLE)
→ VERSION v2 (CANARY)

3 CODEDEPLOY

- Automates canary deployments
- Integrated with Lambda
- Monitors alarms and metrics
- Shifts traffic automatically
- Rolls back on failure

DEPLOYMENT FLOW

CodeDeploy → Lambda Alias → CloudWatch Alarms

4 CANARY10PERCENT5MINUTES

- Built-in canary config
- 10% traffic for 5 minutes
- If healthy, shift to 100%
- Simple and effective
- Default in CodeDeploy

TRAFFIC SHIFT SEQUENCE

10% (5 MIN) → 100% COMPLETE

5 LINEAR TRAFFIC SHIFTING

- Increase traffic in steps
- Example: 10% every 5 minutes
- Customizable intervals
- More control over exposure
- Supported by CodeDeploy

EXAMPLE: LINEAR 10% EVERY 5 MINUTES

10% (5 MIN) → 20% (5 MIN) → 30% (5 MIN) - - - → 100% COMPLETE

6 CLOUDWATCH ALARMS

- Monitor metrics in real time
- Error rate, latency, duration
- Set alarm thresholds
- Trigger rollback on breach
- Essential for safe canary

COMMON METRICS

Errors | Latency | Throttles | Duration

7 API GATEWAY CANARIES

- Enable canary on stage
- Set percentage of traffic
- Route to new stage/version
- Monitor metrics and logs
- Useful for microservices & APIs

CLIENT → API GATEWAY (SOLDENAE)
90% → STAGE (STABLE)
10% → STAGE (CANARY)

8 AUTOMATED ROLLBACK

- Alarms trigger rollback
- Traffic returns to stable version automatically
- No manual intervention
- Minimizes impact
- Zero or low downtime

ROLLBACK FLOW

ALARM TRIGGERED → CODEDEPLOY ROLLS BACK → ALIAS POINTS TO STABLE

9 BEST PRACTICES

- Start with small %
- Validate with real user traffic
- Use meaningful alarms
- Monitor business metrics
- Keep rollback fast and tested
- Communicate during releases

GOLDEN RULE
AUTOMATE EVERYTHING.
MEASURE EVERYTHING.
ROLL BACK INSTANTLY IF NEEDED.

10 AWS CANARY DEPLOYMENT – PRODUCTION TRAFFIC FLOW (LAMBDA EXAMPLE)

USERS → ROUTE 53 (OPTIONAL) → API GATEWAY (CANARY ENABLED) → LAMBDA ALIAS: live

WEIGHTS
v1 (STABLE) 90%
v2 (CANARY) 10%

90% → VERSION v1 (STABLE)
10% → VERSION v2 (CANARY)

HEALTHY → SHIFT TO 100% v2
UNHEALTHY → ROLLBACK TO 100% v1

KEY TAKEAWAYS

- Use Lambda versions + aliases for traffic control
- Start small and shift traffic progressively
- Monitor with CloudWatch and set alarms
- Automate promotions and rollbacks with CodeDeploy
- Canary deployments reduce risk and improve reliability

TRAFFIC SHIFT PATTERNS

- All-at-once (not canary)
- Canary10Percent5Minutes
- Linear (10% every 5 minutes)
- Custom (any step/interval)

PAGE 11

VERIQTA

PROGRESSIVE DELIVERY WITH ARGO ROLLOUTS

DELIVER SMALL. VALIDATE CONTINUOUSLY. PROMOTE WITH CONFIDENCE.

REDUCE RISK | INCREASE RELIABILITY | AUTOMATE PROGRESSION | CONTINUOUS VALIDATION | SAFE RELEASES

1 ROLLOUT RESOURCES

- Rollout is the core resource
- Manages progressive delivery for your application
- Supports canary, blue-green, and experiments
- Defines strategy, steps, traffic routing, and analysis

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
```

2 CANARY STEPS

- Define the canary strategy as a sequence of steps
- Each step can change traffic or pause for evaluation
- Examples: setWeight, pause, analysis

```
steps:
  - setWeight: 10
  - pause: { duration: 5m }
  - setWeight: 50
  - pause: {}
  - setWeight: 100
```

3 SETWEIGHT

- Adjusts traffic weight between stable and canary
- Weight is percentage of traffic sent to canary
- Example: setWeight: 20 sends 20% to canary

4 PAUSE

- Holds the rollout at the current step
- Wait for duration or manual approval
- Allows time for metrics and analysis

```
pause: { duration: 5m }
pause: { }  # wait for manual approval
```

5 ANALYSISTEMPLATES

- Define how to measure health or performance
- Metrics, success/failure conditions, and thresholds
- Reusable across rollouts

Example Metrics:
- Prometheus queries
- Kayenta (statistical analysis)
- Webhooks
- Job / Script checks

6 ANALYSISRUNS

- Executed during rollout at defined steps
- Runs the AnalysisTemplate
- Returns success or failure
- Controls progression

Result: Successful

Result: Failed

7 PROMOTION

- Progresses to the next step automatically if analysis is successful
- Eventually shifts 100% traffic to the new version
- Old ReplicaSet is scaled down

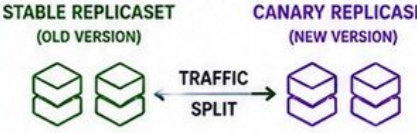
PROMOTE → ALL TRAFFIC TO NEW VERSION

8 ABORT

- Stops the rollout immediately on failure or manual abort
- Traffic returns to stable version
- No downtime rollback

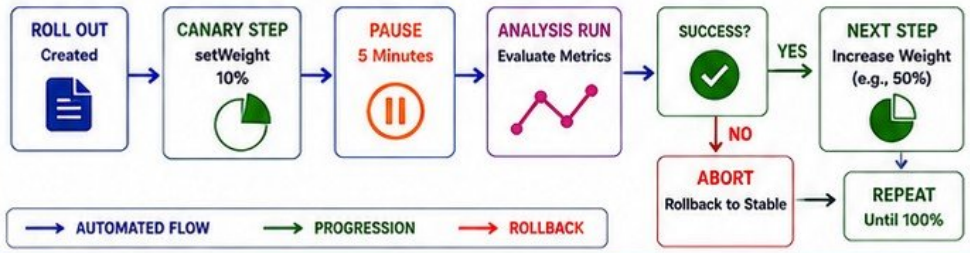
ABORT → ROLLBACK TO STABLE

9 STABLE AND CANARY REPLICASETS



Rollouts manage ReplicaSets and Services to control traffic distribution automatically.

10 AUTOMATED PROGRESSIVE DELIVERY FLOW WITH ARGO ROLLOUTS



KEY BENEFITS

- Reduce deployment risk
- Detect issues early
- Validate with real traffic
- Automate promotion & rollback
- Improve reliability and velocity

VERIQTA

OBSERVABILITY DURING DEPLOYMENT

MEASURE. COMPARE. DECIDE.

WHY IT MATTERS

- Detect problems early
- Reduce deployment risk
- Validate before full rollout
- Protect users and business
- Enable confident releases

1. GOLDEN SIGNALS

ERROR RATE
Percentage of requests resulting in errors.

LATENCY
Time it takes to serve a request.

TRAFFIC
Number of requests served over time.

SATURATION
How "full" your resources are.

THE GOLDEN SIGNALS TELL YOU HOW YOUR SYSTEM IS BEHAVING.

2. ADDITIONAL SIGNALS TO WATCH

APPLICATION METRICS
Business logic metrics from your code. Examples: login success rate, checkout success rate, cache hit rate.

INFRASTRUCTURE METRICS
Health of underlying resources. Examples: CPU, memory, disk I/O, network, DB connections.

BUSINESS KPIs
Metrics that reflect business outcomes. Examples: revenue, orders, sign-ups, active users.

LOGS AND TRACES
Provide context for "why" something happened. Correlate logs and traces across services.

3. COMPARING BASELINE VERSUS CANARY

Always compare the canary version against the stable baseline under the same conditions.

SIGNAL	BASELINE (STABLE)	CANARY (NEW)	WHAT TO LOOK FOR
ERROR RATE	1.2%	1.8%	Spike in errors? If yes, investigate before promoting.
LATENCY (P95)	120 ms	180 ms	Higher latency may degrade user experience.
TRAFFIC (RPS)	5,000	500	Traffic should match the intended canary percentage.
SATURATION	45% CPU	70% CPU	Resource saturation can cause slowdowns or errors.
APPLICATION METRIC	99.5%	98.7%	Business-critical metrics must stay healthy.
BUSINESS KPI	$25,000 /hr	$24,200 /hr	Ensure the new version meets business expectations.

4. BEST PRACTICES

- Define clear success metrics before deployment.
- Set thresholds for automatic promotion or rollback.
- Use the same traffic patterns for fair comparison.
- Monitor continuously during each step.
- Correlate metrics, logs, and traces.
- Validate both technical and business outcomes.
- Document what "good" looks like.

5. REMEMBER

GOOD OBSERVABILITY TURNS DEPLOYMENTS INTO DATA-DRIVEN DECISIONS, NOT GUESSWORK.

PAGE 13

VERIQTA

AUTOMATED DEPLOYMENT GATES

THE RIGHT RELEASE. AT THE RIGHT TIME. EVERY TIME.

WHY GATES MATTER

- Prevent bad releases
- Protect users and business
- Enforce quality automatically
- Reduce manual intervention
- Increase deployment confidence

1. DEPLOYMENT GATE FLOW

2. GATE TYPES

PRE-DEPLOYMENT VALIDATION
Verify artifacts, configs, secrets, permissions, and dependencies.

SMOKE TESTS
Quick checks to confirm the application starts and is reachable.

INTEGRATION TESTS
Validate key user flows and service interactions.

HEALTH CHECKS
Ensure endpoints, services, and infrastructure are healthy.

SLO-BASED GATES
Validate against Service Level Objectives.

ERROR-BUDGET CONSIDERATIONS
Block deployment if error budget is too low.

3. GATE CONDITIONS

CONDITION	EXAMPLE CHECK
METRIC THRESHOLDS	Error rate < 1% P95 latency < 300ms CPU < 70%
AVAILABILITY THRESHOLDS	Success rate > 99.9% Health check pass rate > 99%
PERFORMANCE THRESHOLDS	Throughput within expected range No latency regression
BUSINESS THRESHOLDS	Checkout success rate No drop in conversions No increase in failures
INFRASTRUCTURE THRESHOLDS	Disk < 80% full Connections < limit Queue depth normal

4. AUTOMATED DECISIONS

AUTOMATED PROMOTION

- All gate conditions pass
- Metrics are within safe thresholds
- Error budget is healthy
- No anomalies detected
- Traffic can be increased or cut over

AUTOMATED ABORT

- Any critical gate fails
- Error budget too low
- High error rate or latency
- Health check failures
- Anomalies detected
- Security scan fails
- Infrastructure saturation

5. PREVENTING FALSE-POSITIVE PROMOTION

USE MULTIPLE SIGNALS
Combine metrics, logs, and traces.

USE STABLE WINDOWS
Evaluate metrics long enough to be meaningful.

APPLY STATISTICAL SIGNIFICANCE
Ensure changes are real, not random fluctuations.

SEGMENT TRAFFIC INTELLIGENTLY
Use user cohorts and avoid biased traffic.

VALIDATE BUSINESS OUTCOMES
Check KPIs, not just technical metrics.

USE CONSERVATIVE THRESHOLDS
Prefer safety over aggressive promotion.

6. EXAMPLE GATE CONFIGURATION

```
gate:
  name: canary-promotion
  metrics:
    - name: error_rate
      threshold: "< 1%"
      window: 5m
    - name: p95_latency
      threshold: "< 300ms"
      window: 5m
    - name: availability
      threshold: "> 99.9%"
      window: 5m
evaluation_interval: 30s
evaluation_periods: 10
on_success: promote
on_failure: abort
```

7. BEST PRACTICES

- Define clear success criteria before deployment.
- Automate everything you can.
- Use the same gates across environments.
- Monitor continuously, not just at the end.
- Keep rollback ready and tested.
- Review and tune gates regularly.
- Align gates with user experience and business impact.
- Start conservative, then optimize.

8. REMEMBER

GOOD DEPLOYMENTS ARE NOT LUCK.
THEY ARE THE RESULT OF SMART GATES AND DATA-DRIVEN DECISIONS.

PAGE 14

VERIQTA

ROLLBACK ENGINEERING

DESIGN FOR FAILURE. RECOVER IN SECONDS. PROTECT USERS.

WHY IT MATTERS

- Failures happen
- Fast recovery reduces impact
- Users trust reliable systems
- Rollback is a capability, not a hope

1. WHAT MAKES A RELEASE SAFELY REVERSIBLE

ISOLATED RELEASE
New version runs independently from the old version.

BACKWARD COMPATIBILITY
New code must work with old data and old services.

NO DESTRUCTIVE CHANGES
Avoid irreversible schema, data, and config changes.

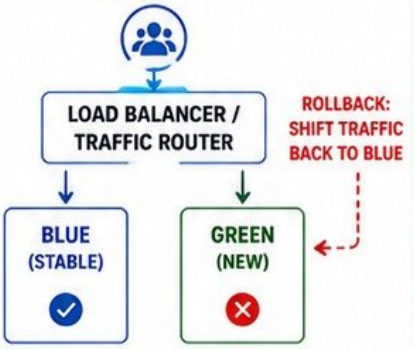
FEATURE FLAGS
Hide functionality until fully validated.

OBSERVABILITY IN PLACE
You can detect issues before users escalate.

AUTOMATED ROLLBACK TESTED
Rollback process is tested, not assumed.

2. INSTANT TRAFFIC ROLLBACK

- Fastest and safest rollback.
- No code changes required.
- Return traffic to the last known good version.



- Works with Blue-Green, Canary, and Progressive Delivery
- Done in seconds

3. APPLICATION ROLLBACK

- Deploy the previous version again.
- Replace containers, pods, or instances with the last good build.
- Ensure artifacts are stored and accessible.

LATEST VERSION (FAILED)
↓
ROLLBACK COMMAND
↓
PREVIOUS VERSION (LAST KNOWN GOOD)

- Ensure build artifacts are immutable and versioned.

4. INFRASTRUCTURE ROLLBACK

- Revert infrastructure to the last stable state.
- Use IaC versioning and state management.
- Roll back changes in DNS, ALB, Auto Scaling, networking, and compute.

- Use Infrastructure as Code for safe, repeatable rollback.

5. CONFIGURATION ROLLBACK

- Restore previous configurations.
- Use version-controlled config and feature flags.
- Roll back environment variables, secrets, and external configs.

- Keep config history. Revert quickly.

6. DATABASE ROLLBACK LIMITATIONS

- Schema changes may not be reversible.
- Data migrations can cause irreversible data loss.
- Rollback may not be possible after destructive operations.
- Application rollback can fail if schema is not backward compatible.
- Use expand-and-contract pattern.

DESIGN DATABASE CHANGES TO BE SAFE, REVERSABLE, AND COMPATIBLE.

7. ROLL-FORWARD VS ROLLBACK

ROLLBACK		ROLL-FORWARD
Return to previous stable state	VS	Move forward with a new fix
Fast and simple in most cases		Do not go backward
Best when issue is known and fixed in previous version		Best when backward state is broken or incompatible
Limited by database and data changes		Requires fast hotfix and re-deployment

PREFER ROLL-FORWARD WHEN BACKWARD STATE IS UNKNOWN, BROKEN, OR EXPENSIVE TO RECREATE.

8. DEFINING ROLLBACK TRIGGERS

ERROR RATE SPIKE
Error rate exceeds threshold

LATENCY DEGRADATION
Latency exceeds threshold

HEALTH CHECK FAILURES
Health check success rate drops

BUSINESS KPI DROP
Key business metrics degrade

SECURITY ALERTS
Security scan or runtime alerts

TRIGGERS MUST BE MEASURABLE, ACTIONABLE, AND AUTOMATED.

9. MAXIMUM ACCEPTABLE RECOVERY TIME (MART)

Define how fast you must recover from a failed deployment.

1. Define MART for your system
2. Measure rollback execution time
3. Continuously optimize for faster recovery
4. Test regularly to meet MART

THE FASTER YOU RECOVER, THE LOWER THE IMPACT.

PAGE 15

VERIQTA

DATABASE CHANGES
DURING PROGRESSIVE DEPLOYMENTS

CHANGE YOUR DATABASE SAFELY. DEPLOY WITH CONFIDENCE.

WHY IT MATTERS

- Database changes are risky
- Rollbacks may not be possible
- Progressive deployments require compatibility
- Design for safe, reversible change

1. BACKWARD-COMPATIBLE SCHEMA CHANGES

- New version must work with old schema.
- Old version must continue to work with new schema.
- Never remove or rename columns in the first release.
- Additive changes are safe.

SAFE EXAMPLES

- ✓ Add new columns (nullable)
- ✓ Add new tables
- ✓ Add new indexes
- ✓ Add new enum values
- ✓ Make columns nullable

2. EXPAND-AND-CONTRACT PATTERN

EXPAND
Additive changes to support new functionality.

↓

MIGRATE / BACKFILL
Populate new columns or tables with required data.

↓

DUAL-WRITE (IF NEEDED)
Write to both old and new structures.

↓

SWITCH READS
Application reads from the new structure.

↓

CONTRACT
Remove old columns, tables, or code in a later release.

3. DUAL-VERSION APPLICATION COMPATIBILITY

- New application version must handle both old and new data.
- Old application must ignore or safely skip new fields.
- Use feature flags to control behavior.
- Avoid breaking changes in APIs and data contracts.

4. DATABASE MIGRATIONS

- Use versioned, repeatable migrations.
- Keep migrations idempotent.
- Test migrations in staging with production-like data.
- Use tools: Flyway, Liquibase, or AWS DMS for complex moves.
- Monitor migration time and locks.

5. FEATURE FLAGS

- Deploy code with features turned off.
- Enable for a small percentage of users.
- Decouple deployment from release.
- Roll back by disabling the flag, not the deployment.

6. DATA BACKFILLS

- Backfills must be safe and resumable.
- Run in small batches.
- Do not impact live traffic.
- Throttle to protect performance.
- Track progress and failures.

7. DESTRUCTIVE SCHEMA CHANGES

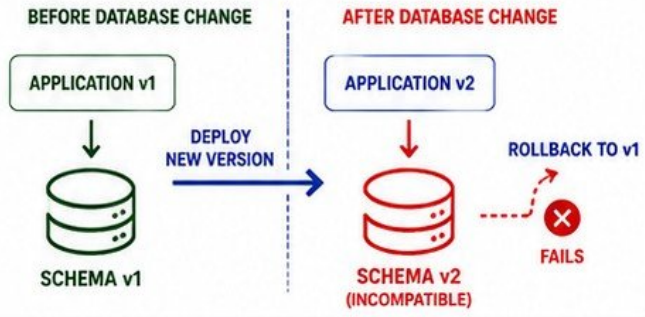
- Dropping or renaming columns is dangerous.
- Requires strict coordination.
- Must be the last step (contract phase).
- Never do destructive changes in the same release as code changes.

EXAMPLES

- ✗ Drop column
- ✗ Rename column
- ✗ Change column type (narrowing)
- ✗ Drop table

8. WHY APPLICATION ROLLBACK CAN FAIL AFTER DATABASE CHANGES

- Schema is no longer backward compatible.
- Old application cannot read new columns.
- Required data was migrated or transformed.
- Dropped or renamed columns break old code.
- Data backfills changed expected behavior.



9. BEST PRACTICES CHECKLIST

- ✓ Design for backward compatibility
- ✓ Use expand-and-contract
- ✓ Use feature flags
- ✓ Test migrations and backfills
- ✓ Monitor performance
- ✓ Automate safely
- ✓ Document every change
- ✓ Validate in staging
- ✓ Plan rollback strategy
- ✓ Communicate changes clearly

10. KEY TAKEAWAY

DATABASE CHANGES ARE PERMANENT.
DESIGN FOR COMPATIBILITY.
DEPLOY SAFELY.
ROLL FORWARD WHEN POSSIBLE.
ROLL BACK ONLY WHEN SAFE.

PAGE 16

VERIQTA

STATEFUL SYSTEMS AND HIDDEN RISKS

STATE DOESN'T ROLL BACK LIKE CODE. PLAN FOR IT.

WHY IT MATTERS

- State breaks simple rollbacks
- Hidden risks cause outages
- Mixed versions create errors
- Protect data, sessions, and users
- Design for compatibility

1. SESSIONS

- User sessions may be stored in memory, DB, or external store.
- Session format changes can break older versions.
- Invalidate or migrate sessions carefully.

Use external session stores (Redis, DB).

2. CACHES

- Cached data may be incompatible.
- Old data can break new versions.
- Cache invalidation is critical.

Version cache keys. Invalidate on deploy.

3. QUEUES

- Messages produced by old versions may not be understood by new versions.
- Changing message schema can cause failures.

Use backward-compatible message schemas.

4. LONG-RUNNING CONNECTIONS

- WebSockets, gRPC, SSE, and DB connections may persist.
- Connections may bind to old versions.
- Can cause inconsistency and errors.

Drain connections before switching traffic.

5. STATEFUL SERVICES

- Databases, file stores, search indexes, and other stateful systems cannot be rolled back easily.
- Plan changes carefully.

Understand recovery time and limitations.

6. VERSION COMPATIBILITY

- New and old versions must work together.
- APIs, data, and contracts must remain compatible.
- Plan compatibility windows for deployments.

Maintain backward compatibility.

7. SCHEMA MISMATCHES

- Schema changes can break old code.
- Removing or renaming fields is dangerous.
- Additive changes are safer.

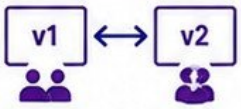
Use expand-and-contract for schema changes.

8. STICKY SESSIONS

- Users may be stuck to one instance or version.
- Breaks even traffic distribution.
- Can hide issues in canary releases.

Avoid stickiness or use consistent hashing.

9. MIXED-VERSION TRAFFIC



- Mixed versions can cause unpredictable behavior.
- Cross-version calls may fail.
- Data may be read/written in incompatible formats.

Test mixed-version scenarios thoroughly.

10. PREVENTING CROSS-VERSION FAILURES

- Design for compatibility.
- Use feature flags.
- Version your APIs.
- Validate inputs strictly.
- Monitor dependencies.

Compatibility + testing = safe progressive delivery.

11. BEST PRACTICES FOR STATEFUL SYSTEMS

DESIGN FOR BACKWARD COMPATIBILITY

Keep old and new versions working together.

EXTERNALIZE STATE

Use external stores for sessions, caches, and queues.

USE EXPAND-AND-CONTRACT PATTERN

Add first, migrate, then remove in a later release.

USE FEATURE FLAGS

Control behavior without immediate deployments.

MONITOR STATE DURING DEPLOYMENT

Track sessions, queue depth, cache hit rate, and errors.

TEST LIKE PRODUCTION

Include real data, traffic patterns, and failure scenarios.

12. COMMON HIDDEN RISKS

- Session loss after deployment
- Stale cache serving incorrect data
- Old workers processing new messages
- Long-lived connections never refreshed
- Database locks during schema migration
- Background jobs running against wrong version
- Data corruption from schema mismatch

13. KEY TAKEAWAY

STATE DOESN'T ROLL BACK.
PLAN FOR COMPATIBILITY.
OBSERVE EVERYTHING.
PREVENT CROSS-VERSION FAILURES.
DELIVER CONFIDENTLY.

VERIQTA

SECURITY AND COMPLIANCE

SECURE EVERY RELEASE. PROTECT EVERY ENVIRONMENT.

WHY IT MATTERS

- Prevent breaches and tampering
- Protect sensitive data and secrets
- Meet compliance requirements
- Ensure accountability and traceability
- Enable safe and reliable deployments

1 ARTIFACT INTEGRITY

- Ensure artifacts are authentic and unmodified.
- Detect tampering early.
- Verify checksums and signatures.

Verify integrity before every deployment.

2 IMAGE SIGNING

- Sign container images with trusted keys.
- Prevent unauthorized or compromised images.
- Enforce signature verification.

Only run signed and trusted images.

3 SECRETS MANAGEMENT

- Never hardcode secrets.
- Store in a secure secret manager.
- Rotate secrets regularly.

Secure secrets from dev to prod.

4 IAM AND DEPLOYMENT IDENTITIES

- Use least-privilege IAM policies.
- Use dedicated deployment roles and federated identities.
- Avoid long-lived credentials.

Minimize permissions. Eliminate shared accounts.

5 APPROVAL GATES

- Require human or policy approvals at key stages.
- Enforce quality, security, and compliance checks.
- Block risky changes.

No approval, no production.

6 SEPARATION OF DUTIES

- Separate who builds, deploys, and approves.
- Reduce risk of fraud or accidental changes.
- Enforce role boundaries.

Different roles for different responsibilities.

7 AUDIT TRAILS

- Log every action and change.
- Track who did what, when, and where.
- Protect logs from tampering.

If it isn't logged, it didn't happen.

8 PRODUCTION ACCESS

- Restrict direct access to production systems.
- Use bastion hosts or SSM.
- Enforce MFA and just-in-time access.

Access only when necessary and approved.

9 SUPPLY-CHAIN CONTROLS

- Scan dependencies and base images.
- Use trusted registries and repositories.
- Monitor for known vulnerabilities.

Trust, verify, and continuously monitor.

10 EMERGENCY DEPLOYMENT PROCEDURES

- Have a documented break-glass process.
- Enable rapid rollback or hotfix deployment.
- Communicate and review after incidents.

Be prepared before an emergency happens.

SECURITY AND COMPLIANCE BEST PRACTICES

SECURE BY DESIGN
Build security into your pipelines and applications.

CONTINUOUS VALIDATION
Scan, test, and validate at every stage.

LEAST PRIVILEGE
Grant only the minimum access required.

MONITOR EVERYTHING
Visibility is your best defense.

AUTOMATE COMPLIANCE
Use policy-as-code and automated checks.

TRAIN YOUR TEAM
Security and compliance are everyone's job.

STRONG SECURITY BUILDS TRUST.
COMPLIANT DEPLOYMENTS PROTECT YOUR BUSINESS.

PAGE 18

VERIQTA

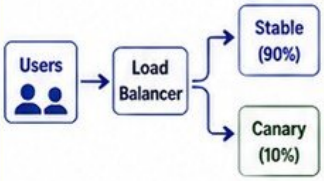
PRODUCTION FAILURE SCENARIOS

REAL PROBLEMS. REAL IMPACT. REAL SOLUTIONS.

WHY IT MATTERS

- Fail fast, not silently
- Reduce mean time to recovery
- Protect users and revenue
- Learn and improve continuously
- Build reliable delivery systems

1 CANARY METRICS LOOK HEALTHY WHILE USERS FAIL

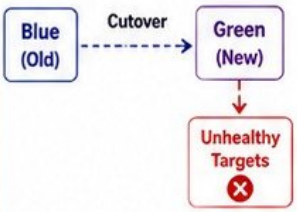


- Synthetic checks pass but real user flows fail.
- Error budget not impacted.
- Issues in specific user cohorts or regions.

WHAT TO DO

- Validate real user journeys.
- Use RUM and business KPIs.
- Add targeted canary tests.
- Increase signal diversity.

2 BLUE-GREEN CUTOVER SENDS TRAFFIC TO UNHEALTHY TARGETS

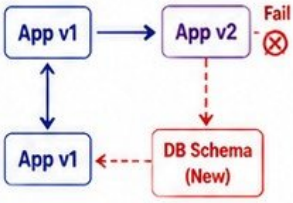


- Health checks misconfigured.
- Targets fail after cutover.
- All traffic hits failing version.

WHAT TO DO

- Validate targets before cutover.
- Use automated pre-cutover tests.
- Enable connection draining.
- Keep blue environment warm.

3 ROLLBACK SUCCEEDS BUT DATABASE REMAINS INCOMPATIBLE

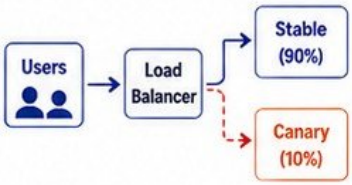


- App rolled back successfully.
- Schema or data changes persist.
- Old version breaks on new schema.

WHAT TO DO

- Use backward-compatible schema.
- Avoid destructive changes.
- Plan rollback strategy for DB.
- Use expand-and-contract pattern.

4 STICKY SESSIONS HIDE CANARY FAILURES

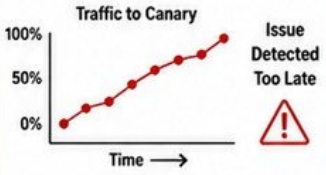


- Sticky sessions keep users on one version.
- Failures hidden from metrics.
- Issues appear after full rollout.

WHAT TO DO

- Test with stickiness disabled.
- Use diverse session routing.
- Monitor per-version metrics.
- Simulate new user sessions.

5 MONITORING DELAY CAUSES EXCESSIVE TRAFFIC EXPOSURE

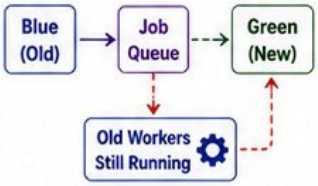


- Alerts fire too late.
- Slow dashboards or high aggregation windows.
- Too much bad traffic.

WHAT TO DO

- Use real-time or near real-time metrics.
- Set aggressive alert thresholds.
- Automate rollback on alarm.
- Monitor leading indicators.

6 OLD ENVIRONMENT RECEIVES BACKGROUND JOBS



- Background workers not stopped.
- Jobs processed by wrong version.
- Data corruption or duplicates.

WHAT TO DO

- Drain and stop old workers.
- Use version-aware job queues.
- Validate consumer versions.
- Monitor job success and errors.

7 DEBUGGING SEQUENCE FOR EACH FAILURE

1 IDENTIFY THE IMPACT	Who is affected? What is failing? User vs. system symptoms.
2 CHECK METRICS	Compare baseline vs. canary/green. Look for anomalies.
3 REVIEW LOGS & TRACES	Find errors, timeouts, exceptions, slow calls.
4 VERIFY ROUTING	Confirm traffic distribution, stickiness, and health checks.
5 CHECK DEPENDENCIES	Database, cache, queue, external APIs, credentials, limits.
6 REPRODUCE	Recreate issue in controlled environment if possible.
7 DECIDE & ACT	Rollback, fix forward, or mitigate. Communicate clearly.

PREVENTION PRINCIPLES

TEST LIKE PRODUCTION
Real data, real traffic patterns, real users.

OBSERVE EVERYTHING
Metrics, logs, traces, and KPIs.

AUTOMATE SAFETY
Health checks, alarms, rollbacks.

LIMIT BLAST RADIUS
Small steps, quick rollback options.

LEARN AND IMPROVE
Post-incident reviews and runbook updates.

KEY TAKEAWAY

FAILURES WILL HAPPEN.
PREPARE FOR THEM.
DETECT EARLY.
RESPOND FAST.
IMPROVE ALWAYS.

PAGE 19

VERIQTA

ENTERPRISE DEPLOYMENT ARCHITECTURE

END-TO-END DEPLOYMENT PIPELINE WITH BLUE-GREEN & CANARY STRATEGIES

ARCHITECTURE PRINCIPLES

- Secure by design
- Automated and repeatable
- Progressive and safe delivery
- Observable and measurable
- Fast rollback and recovery

1 GIT REPOSITORY

- Source code
- Version control
- Branch protection
- Webhooks

2 PULL REQUEST VALIDATION

- Code review
- Static analysis
- Lint & format
- PR checks

3 BUILD AND TEST

- Build application
- Unit tests
- Integration tests
- Code coverage

4 SECURITY SCANNING

- SAST scanning
- Dependency scan
- Container scan
- Secrets scan

5 ARTIFACT REGISTRY

- Store artifacts
- Container images
- Versioned
- Immutable

6 STAGING DEPLOYMENT

- Deploy to staging
- Smoke tests
- Integration tests
- User acceptance

7 PRODUCTION DEPLOYMENT CONTROLLER

- Deployment strategy
- Blue-Green / Canary
- Rollout orchestration
- Policy enforcement

8 PROGRESSIVE TRAFFIC SHIFT

1% 5% 25% 50% 100%

- Weighted routing
- Canary cohorts
- Traffic shifting

9 AUTOMATED ANALYSIS

- Metrics evaluation
- SLO / Error budget
- Anomaly detection
- Success / Failure

10 APPROVAL GATES

- Manual approval
- Policy checks
- Change windows
- Compliance

11 OBSERVABILITY

- Metrics (Golden Signals)
- Logs and traces
- Alerts and dashboards
- Synthetic monitoring

12 PROMOTION OR ROLLBACK

PROMOTE

- Full traffic
- Decommission old
- Cleanup resources
- Update records

ROLLBACK

- Stop traffic shift
- Revert routing
- Restore previous
- Investigate issue

DEPLOYMENT STRATEGIES

BLUE-GREEN (B ⇄ G)

CANARY

Choose strategy based on risk, application type, and business needs.

CONTINUOUS FEEDBACK LOOP

DEVELOPERS — Feedback from metrics, errors, and user reports.

IMPROVE — Refine code, tests, and processes.

RELEASE BETTER — Faster delivery with higher quality.

KEY OUTCOMES

- Lower deployment risk
- Faster time to market
- Higher reliability
- Better visibility
- Confident releases

VERIQTA

AUTOMATE. OBSERVE. VALIDATE. APPROVE. DELIVER.
SAFE DEPLOYMENTS. HAPPY USERS. STRONG BUSINESS.

/VERIQTA /VERIQTA /VERIQTA /VERIQTA /VERIQTA /VERIQTA /VERIQTA

PAGE 20

VERIQTA

PRODUCTION DEPLOYMENT DECISION FRAMEWORK

CHOOSE THE RIGHT STRATEGY. CONTROL RISK. DELIVER WITH CONFIDENCE.

DECISION GOAL

- Minimize risk
- Protect users and revenue
- Meet SLOs and compliance
- Optimize for speed and reliability
- Make the right trade-offs

1. BLUE-GREEN SELECTION CRITERIA

B ⇄ G

- Need near-zero downtime
- Easy and fast rollback
- Full environment parity possible
- Stateless or backward-compatible systems
- High risk of deployment failure
- Adequate infrastructure capacity for two environments
- Database changes are backward compatible

BEST FOR STABILITY AND FAST ROLLBACKS

2. CANARY SELECTION CRITERIA

- Need gradual exposure
- Validate in real production
- Can isolate impact to a subset of users
- Strong observability in place
- Low to medium blast radius
- Services are stateless or can handle mixed versions
- Automated analysis and rollback available

BEST FOR LEARNING AND VALIDATION

3. ROLLING DEPLOYMENT CONSIDERATIONS

- Small, incremental updates
- Lower infrastructure cost
- Slight risk during rollout
- Requires application tolerance for mixed versions
- Use maxUnavailable/maxSurge carefully
- Not ideal for high-risk changes

BEST FOR LOW-RISK, STATELESS CHANGES

4. FEATURE FLAGS VS DEPLOYMENT STRATEGIES

FEATURE FLAGS	DEPLOYMENT STRATEGIES
Decouple release from deploy	Manage infrastructure and traffic risk
Enable/disable features dynamically	Control version exposure
Target users or segments	Handle failures at infrastructure level
Reduce need for multiple deployments	Required for breaking changes
Great for experimentation	

USE TOGETHER FOR MAXIMUM CONTROL

5. RISK CLASSIFICATION

HIGH RISK	Breaking changes Critical user flows New infrastructure High impact	→ Use Blue-Green or Canary
MEDIUM RISK	New features Partial user impact Some dependencies	→ Use Canary or Rolling
LOW RISK	Minor changes Internal tools Non-critical paths	→ Use Rolling Deployment

6. BLAST-RADIUS LIMITS

- Limit exposure to a small % of users
- Use service isolation and micro-segmentation
- Define maximum acceptable impact
- Monitor real impact during rollout

KEEP THE IMPACT SMALL AND CONTROLLED

7. SLO AND ROLLBACK REQUIREMENTS

- Define SLOs and error budgets
- Set clear success metrics
- Automate rollback on SLO breach
- Define Maximum Acceptable Recovery Time (MART)
- Test rollback process regularly

IF IT'S NOT AUTOMATED, IT'S NOT RELIABLE

8. COST VS RELIABILITY TRADE-OFFS

COST FACTORS	RELIABILITY BENEFITS
Duplicate environments	Lower downtime risk
Extra compute capacity	Faster rollback
Observability tools	Real user validation
Longer testing windows	Higher change confidence
Operational complexity	Protects revenue and reputation

INVEST FOR RELIABILITY. SAVE COSTS ELSEWHERE.

9. PRE-PRODUCTION READINESS CHECKLIST

- [x] Code reviewed and approved
- [x] Automated tests passing
- [x] Security scans clean
- [x] Artifacts signed and verified
- [x] Infra as Code validated
- [x] Configurations verified
- [x] Secrets and permissions ready
- [x] Observability and alerts ready
- [x] Runbooks and rollback plan ready
- [x] Stakeholders informed

NO CHECKLIST, NO DEPLOYMENT

10. FINAL PRODUCTION DEPLOYMENT DECISION TREE